

TASK 2: Environment Design Note:

Document the environment with a short design note (1 to 2 pages) covering the action space, the observation format, the reward structure, the obvious reward-hacking failure modes, and what you would monitor if a real training run were using it.

Action Space:

The agent sends JSON containing a Bash command, a reasoning string, and an is_complete flag.

Every command runs in a brand-new subshell (docker exec bash -c), so state changes like cd or export do not persist across steps.

Observation Format:

Output contains stdout, stderr (capped to 200 characters), and an exit code. Empty output defaults to "(no output)".

Past inputs and outputs are saved in the LLM's conversation history as context.

I also track below execution details in the trajectory:

- step_num
- command / action
- reasoning
- is_complete
- observation
- exit_code
- binary_reward & shaped_reward

Reward Structure:

Base Reward: 1.0 if /tests/test.sh passes at the end of the episode, 0.0 otherwise.

Step Penalty: Free for steps 1–5, then incurs a penalty of 0.01 per step

Obvious Reward-Hacking Vulnerabilities:

- Overwriting Reward Files: /tests and /logs are mounted as read-write, allowing the agent to run echo "1.0" > /logs/verifier/reward.txt or edit /tests/test.sh to pass without solving the task.
- Quitting Early: Failing at step 15 yields a negative score (e.g. -0.10), while quitting at step 1 yields 0.0. Agents can learn to immediately surrender on hard tasks.
- Hanging Commands: Interactive commands (like top or vim) freeze execution indefinitely without timeouts.

Metrics I would monitor:

Test tamper count: Detects if /tests or /logs were modified.

Early quit rate: Tracks how often the agent quits on step 1 to dodge penalties.

Repeated_commands : Flag when the agent gets trapped repeating commands like cd.

Step duration seconds: Spikes indicate hanging or frozen commands.

And look out for other adversarial actions.